

Cahier des Charges

Marketplace Intelligente

Projet Trans DIC2 — École Polytechnique de Thiès
Mamadou Falilou Diaw - Abdoulaye Sy Ndaw
Encadreurs : Pr Gueye, Pr Ciss

1. Contexte et Objectifs

1.1 Contexte

Ce projet s'inscrit dans la continuité des travaux engagés en DIC1, au cours desquels une première version de la marketplace a été conçue et implémentée. Cette base technique comprend une architecture microservices Spring Boot composée de sept services opérationnels : **product-service**, **order-service**, **cart-service**, **avis-service**, **customer-service**, **authentication-service** et un **gateway-service** appuyé sur Eureka pour la découverte de services.

La présente phase vise à élever cette infrastructure en une plateforme intelligente, en y intégrant des capacités d'analyse de données et d'intelligence artificielle.

1.2 Objectifs du projet

- Améliorer l'expérience d'achat par une recherche sémantique et des recommandations personnalisées.
- Optimiser la gestion des stocks des vendeurs grâce à des prévisions de demande et des alertes automatiques.
- Détecter et anticiper les tendances du marché pour guider les décisions commerciales.
- Garantir l'intégrité de la plateforme par la détection automatique d'activités frauduleuses.

2. Acteurs du Système

La plateforme distingue trois types d'acteurs, chacun disposant d'une interface et de droits adaptés à son rôle.

Acteur	Description	Fonctions principales
Acheteur	Utilisateur final qui consulte et achète des produits.	Recherche, recommandations, achat, suivi de commandes, rédaction d'avis.
Vendeur	Marchand qui commercialise des produits et gère son activité.	Gestion du catalogue, suivi des stocks, statistiques de ventes, alertes et suggestions.
Administrateur	Opérateur chargé de la supervision globale.	Gestion des comptes, supervision des transactions, statistiques globales, modération.

3. Fonctionnalités Principales

3.1 Recherche Sémantique Intelligente

La recherche sémantique va au-delà du matching de mots-clés : elle comprend l'intention derrière la requête et retourne des résultats pertinents même si les termes exacts ne sont pas utilisés.

Exemple : "chaussure sport homme" → baskets, sneakers, running shoes sans qu'aucun de ces mots ne figure dans la requête.

Fonctionnalités détaillées

- **Auto-complétion** : suggestions en temps réel dès les premières frappes.
- **Correction orthographique** : tolérance aux fautes de frappe via fuzzy search.
- **Compréhension sémantique** : correspondance par vecteurs d'embedding entre la requête et les fiches produits.
- **Filtres combinés** : résultats sémantiques croisés avec des critères structurés (prix, catégorie, disponibilité).
- **Support multilingue** : modèle d'embedding multilingue pour couvrir plusieurs marchés.
- **Classement par pertinence** : score composite combinant similarité sémantique, popularité et fraîcheur.

Stack technique

Composant	Outil	Rôle
Modèle d'embedding	sentence-transformers	Génération des vecteurs textuels
Base vectorielle	Qdrant / Weaviate	Stockage et requêtage des embeddings
Correction ortho.	SymSpell / ES fuzzy	Tolérance aux erreurs de saisie
Auto-complétion	Elasticsearch / Redis	Index de suggestions temps réel

3.2 Recommandation de Produits par IA

Le moteur de recommandation combine trois approches complémentaires pour couvrir tous les profils d'utilisateurs, du nouvel arrivant au client régulier.

a) Produits similaires — Filtrage basé sur le contenu

Repose sur les embeddings calculés pour la recherche sémantique. Pour chaque produit consulté, les N produits vectoriellement les plus proches sont retournés, enrichis de contraintes structurées (même tranche de prix, même marque, même catégorie). Aucune donnée utilisateur n'est requise : ce module est disponible dès le lancement.

b) Recommandations personnalisées — Filtrage collaboratif

Repose sur la matrice d'interactions utilisateurs × produits (vues, clics, ajouts au panier, achats). Une décomposition matricielle (SVD ou ALS) prédit les scores d'affinité pour les produits non encore consultés.

- Bibliothèques : **Implicit** (grands volumes), **Surprise** (volumes réduits).
- Cold start : les nouveaux utilisateurs reçoivent d'abord les produits populaires, puis basculent vers le collaboratif après quelques interactions.

c) Produits populaires

Score de popularité calculé sur fenêtre glissante (7 j / 30 j) pondéré par un facteur de décroissance temporelle. Segmenté par catégorie pour éviter la sur-représentation d'un seul domaine.

Architecture hybride

Score final = $\alpha \times \text{score_sémantique} + \beta \times \text{score_collaboratif} + \gamma \times \text{score_popularité}$

Les poids α , β , γ s'ajustent dynamiquement : nouvel utilisateur → plus de popularité ; utilisateur fidèle → plus de collaboratif.

Approche	Données requises	Disponibilité
Produits similaires	Catalogue uniquement	Dès le lancement
Filtrage collaboratif	Historique d'interactions	Dès ~1 000 utilisateurs actifs
Produits populaires	Volume de ventes et vues	Dès les premières transactions

3.3 Optimisation du Stock

Ce module aide les vendeurs à maintenir un niveau de stock optimal, évitant les ruptures qui font perdre des ventes et le sur-stockage qui immobilise de la trésorerie.

a) Tableau de bord des stocks

Chaque produit dispose d'indicateurs visuels codés par couleur (vert / orange / rouge) basés sur deux seuils configurables par le vendeur.

```
jours_restants = stock_actuel / (ventes_30j / 30)
```

b) Alertes automatiques

- Notification (email, dashboard) déclenchée quand $\text{stock_actuel} \leq \text{seuil_alerte}$.
- Quantité suggérée : $(\text{délai_livraison} \times \text{ventes_moy.}) + \text{stock_sécurité}$.
- Priorisation des alertes : ruptures immédiates d'abord, puis ruptures prévues sous 7 jours.

c) Prédiction de la demande par IA

Volume de données	Méthode	Précision
< 4 semaines	Moyenne mobile pondérée	Indicative
4 – 12 semaines	Prophet (Meta)	Bonne
> 12 semaines	LightGBM (features temporelles)	Haute

d) Suggestions de promotions

- Détection des produits dormants : aucune vente depuis X jours avec stock disponible.
- Proposition automatique d'une remise estimée à partir de l'élasticité prix de produits similaires.
- Option : suggestion de bundles (regroupement de produits complémentaires peu vendus).

3.4 Prédiction des Tendances

L'objectif est d'identifier les produits et catégories susceptibles de connaître une croissance significative avant que celle-ci ne soit visible dans les ventes.

a) Collecte des signaux

Événement	Description	Poids
product_view	Consultation d'une fiche produit	Faible
search_query	Requête de recherche	Moyen
wishlist_add	Ajout liste de souhaits	Moyen

Événement	Description	Poids
add_to_cart	Ajout au panier	Fort
purchase	Achat confirmé	Très fort

b) Score de tendance

$$\text{score} = (\text{evenements_7j} - \text{evenements_7j_precedents}) / (\text{evenements_7j_precedents} + 1)$$

Un facteur de décroissance temporelle est appliqué : les événements récents pèsent plus lourd que les anciens.

c) Détection des signaux faibles

- Surveillance des requêtes : tout terme dont la fréquence progresse anormalement est signalé.
- Détection d'anomalies par z-score : $z > 2,0$ sur une série temporelle déclenche un signal d'alerte.
- Croisement optionnel avec Google Trends (via pytrends) pour valider les signaux internes.

d) Affichage sur la plateforme

- Section *Tendances du moment* en page d'accueil : top 10 produits par score de tendance, segmentés par catégorie.
- Badge *En hausse* ou *Populaire cette semaine* sur les fiches produit.
- Rapport hebdomadaire automatique pour les vendeurs : catégories en progression avec recommandations d'action.

4. Architecture Technique

La plateforme adopte une architecture microservices organisée en couches. Chaque domaine métier est un service indépendant. Les modules IA communiquent de façon asynchrone via un broker de messages, sans impacter l'expérience utilisateur.

4.1 Vue d'ensemble

```
FRONTEND (React + Vite) ↔ API GATEWAY ↔ Microservices Métier ↓ MESSAGE BROKER  
(Kafka) ↓ Workers IA (Python / Celery) ↓ DATA LAYER
```

4.2 Couches de l'Architecture

Frontend

- **Acheteur / Vendeur / Admin** : React 18 + Vite — rendu SPA.
- Communication REST pour les données classiques, WebSocket pour les alertes temps réel.
- Tableaux de bord vendeur avec graphiques interactifs (Recharts).

API Gateway

- Point d'entrée unique pour tous les appels frontend.
- Authentification : JWT + refresh tokens, OAuth2 pour la connexion sociale.
- Rate limiting : protection contre les abus et les attaques par déni de service.
- Outil recommandé : Kong, Nginx ou FastAPI selon la taille de l'équipe.

Microservices Métier

Service	Responsabilité	Base de données
Product Service	CRUD catalogue, fiches produits et embeddings	MongoDB + Qdrant
Order Service	Gestion des commandes et des paiements	MongoDB
User Service	Comptes, profils, historique d'interactions	PostgreSQL
Search Service	Recherche sémantique et auto-complétion	Qdrant + Elasticsearch

Message Broker — Kafka

Kafka est la colonne vertébrale des modules IA. Chaque action utilisateur publie un événement consommé de façon asynchrone par les workers :

- **user.product_viewed** → alimente les recommandations collaboratives.
- **order.completed** → met à jour les stocks, déclenche le réentraînement des modèles.
- **search.query_made** → alimente le module de tendances.
- **user.review_posted** → déclenche l'analyse de fraude.

Data Layer

Stockage	Usage principal
PostgreSQL	Gestion des utilisateurs, authentification, rôles
MongoDB	Données métier (produits, commandes, catalogue)

Stockage	Usage principal
Redis	Cache des recommandations, sessions, rate limiting
Qdrant	Embeddings vectoriels pour la recherche et les recommandations
ClickHouse (DW)	Historique d'événements pour l'entraînement et l'analyse IA